

RECAST - 面向大型工业软件的 智能检索与自适应开发平台

项目说明书

RECAST

项目团队：codeX

2026 年 9 月

目录

第一章 项目概述	1
1.1 项目简介	1
1.2 项目定位	1
1.3 核心应用场景	2
1.4 项目建设目标	2
1.5 项目总体能力	3
1.6 项目当前阶段	3
第二章 背景与需求分析	4
2.1 行业背景与国家战略需求	4
2.1.1 国产软件与操作系统生态发展	4
2.1.2 AI Coding 技术发展趋势	4
2.1.3 超大规模软件工程智能化需求	5
2.2 命题企业与业务背景	5
2.2.1 命题企业相关业务背景	5
2.2.2 大型软件工程与代码迁移场景	5
2.3 命题需求分析	6
2.3.1 命题总体目标	6
2.3.2 命题指标拆解	6
2.3.3 命题本质分析	6
2.4 典型业务场景	7
2.4.1 功能定位场景	7
2.4.2 代码迁移场景	7
2.4.3 大型工程修改场景	7
2.5 现有 AI Coding 工作方式分析	7
2.6 现有技术方案及其局限	8

目录	II
2.6.1 关键词与全文检索	8
2.6.2 LSP 与符号检索	8
2.6.3 向量语义检索	8
2.6.4 传统 RAG	8
2.6.5 Coding Agent 多轮探索	9
2.7 核心技术难点总结	9
2.7.1 超大规模代码仓建模	9
2.7.2 跨语言跨模块依赖分析	9
2.7.3 高精度低延迟代码检索	9
2.7.4 任务级上下文构建	10
第三章 市场现状与竞品分析	11
3.1 市场供给现状	11
3.2 主要产品对比	11
3.3 竞争判断	12
3.4 项目差异化方向	12
第四章 面向功能理解的代码检索框架	14
4.1 从功能需求到分散实现	14
4.2 离线建模与在线检索的协同架构	14
4.3 从功能定位到开发执行的技术主线	15
第五章 依赖约束的自适应建模与增量索引	16
5.1 以统一符号模型连接多语言源码	16
5.2 以有界流水线控制大仓建模的内存增长	17
5.3 面向功能定位的依赖约束自适应模块划分	17
5.4 以构建上下文恢复跨语言调用路径	18
5.5 按变化影响范围更新模型与索引	19
第六章 任务驱动的分层检索与上下文构建	21
6.1 以职责层级缩小搜索范围	21
6.2 融合声明、正文与模块语义的混合召回	21
6.3 通过候选按需读取降低在线开销	22
6.4 沿任务相关依赖补全实现路径	23
6.5 以必要覆盖和冗余消除构建精简上下文	24

第七章 基于代码上下文的 Agent 开发协同	25
7.1 将任务检索接入 Agent 的分析与修改流程	25
7.2 从参考实现到目标工程的适配规划	25
7.3 以行为验证连接修改结果与任务需求	26
第八章 项目进展与规划	27
8.1 项目研发起点	27
8.2 项目开发时间线	27
8.3 版本迭代过程	28
8.4 当前系统完成情况	28
8.5 当前产品形态	29
8.6 阶段性测试环境	29
8.7 阶段性验证结果	29
8.7.1 大规模工程离线建模	29
8.7.2 平均检索响应时间	30
8.7.3 依赖分析	30
8.8 经济效益分析	31
8.8.1 效益来源与测算方法	31
8.8.2 降低代码定位与理解成本	33
8.8.3 提升代码资产复用率，减少重复开发	34
8.8.4 降低 AI Coding 的上下文与探索成本	34
8.8.5 缩短大型迁移与适配周期	35
8.8.6 平台投入成本与净收益	36
8.8.7 测算验证方案与不确定性说明	38
8.9 下一阶段开发与验证计划	41
8.9.1 大仓建模与增量维护	41
8.9.2 跨语言依赖分析	41
8.9.3 检索与上下文优化	41
8.9.4 产品流程与企业验证	42
第九章 项目团队与组织分工	43
9.1 团队总体情况	43
9.2 团队组织与任务分工	43
9.2.1 指导教师	44

9.2.2 学生团队	45
9.3 师生协同与项目组织	47

第一章 项目概述

1.1 项目简介

RECAST 是一套面向大型工业软件与企业级代码资产的智能理解、检索与开发支撑平台。系统服务于百万至千万行级、多仓库、多语言软件工程，通过增量索引、依赖分析、混合检索和任务级上下文构建，为开发人员及 AI Coding 智能体提供准确、完整且规模适当的工程上下文。

项目作为现有 AI Coding 工具的大型工程理解支撑层，重点处理代码规模、语言差异、工程依赖和任务定位等问题，再将筛选后的候选代码、关键符号及相关依赖交给智能体分析和修改。通过这一方式，可以减少智能体在超大工程中反复执行搜索、读取和依赖追踪的次数，降低无关上下文对生成结果的干扰。

RECAST 的核心目标是为 AI Coding 提供大型工业软件的工程级代码理解与检索能力，支撑复杂工程中的功能定位、代码修改、软件迁移和系统维护。

1.2 项目定位

RECAST 定位为大型工业软件代码资产与 AI Coding 工具之间的工程理解基础设施。

产品首先接入企业代码仓库，建立多粒度代码表示、模块知识结构和工程依赖图谱；接到查询后，系统通过关键词、符号、语义和依赖关系联合检索候选代码；随后按照任务需要补齐接口、类型、配置和测试等必要信息，形成面向当前任务的工程上下文，再通过标准化接口交付给 Coding Agent 使用。

与单纯的代码搜索工具相比，RECAST 不只回答“哪个文件出现了这个字符串”；与普通向量 RAG 相比，系统不只寻找语义相似代码；与通用 Coding Agent 相比，RECAST 将超大仓库解析、索引、依赖恢复和候选压缩等稳定工程能力前置，为智能体提供更高质量的任务入口和代码证据。

1.3 核心应用场景

RECAST 主要面向大型制造、能源、交通、金融及央国企的软件研发部门，也适用于承担遗留系统改造的软件集成商和维护多仓库产品线的平台团队。

主要应用场景包括：

- 在大型代码仓中，根据自然语言需求快速定位相关模块、文件、类型和函数；
- 在多套业务系统中识别重复功能，为系统精简和能力合并提供依据；
- 在语言、框架或平台升级前定位可复用模块，并恢复迁移所需依赖；
- 根据需求描述、接口、代码片段或错误日志检索企业内部已有实现；
- 在修改公共接口或基础模块前分析跨仓库、跨模块影响范围；
- 在 Java、ArkTS、Kotlin/KMP、C/C++ 等多语言工程中恢复跨语言调用和绑定关系；
- 为 AI Coding 智能体提供带有来源、关系、优先级和代码证据的工程上下文。

1.4 项目建设目标

本项目面向超大规模工业软件工程中的代码理解与 AI Coding 场景，建设一套能够支持多语言大型工程建库、代码理解、任务检索、依赖分析和上下文构建的基础平台。

项目重点围绕以下目标展开：

1. 面向百万至千万行级大型软件工程建立可扩展、可增量维护的代码索引；
2. 支持 Java、ArkTS、Kotlin Multiplatform、C/C++ 等多语言代码理解；
3. 建立跨文件、跨模块和跨语言的统一工程依赖表示；
4. 支持自然语言到代码的 NLP2Code 检索能力，快速定位与开发任务真正相关的代码；
5. 在有限时延和计算资源条件下兼顾检索召回率、准确率和执行效率；
6. 根据开发任务自动组织必要代码及依赖，形成可直接提供给 Coding Agent 的任务级上下文；
7. 通过 MCP、HTTP、IDE 等方式与不同 AI Coding 工具进行集成。

1.5 项目总体能力

RECAST 总体能力覆盖代码资产接入、代码建库、代码理解、依赖恢复、智能检索、上下文构建和 Agent 接入等环节。

系统对多个仓库、分支和版本进行统一管理，识别源码、构建文件、接口描述、配置和测试等工程信息；进一步建立仓库、模块、文件、类型、函数和代码块等多粒度表示，并结合全文索引、符号索引、向量索引和工程依赖关系形成统一的检索基础。

在在线查询阶段，系统可接收自然语言需求、代码片段、符号名称、接口签名、异常日志和文件路径等输入，通过词法、符号、语义和结构信息联合召回候选，再依据接口、模块和工程关系进行筛选与重排。对于需要进一步理解或修改的任务，系统还可以沿依赖关系补充必要源码、接口、配置及测试证据，为下游 Coding Agent 提供结构化任务上下文。

1.6 项目当前阶段

目前，项目已完成前期需求分析、总体技术路线设计和平台原型搭建，代码理解、索引构建、智能检索、依赖分析及上下文组织等主要功能已形成基本完整的系统链路，可通过现有 Demo 完成从代码仓接入、工程分析到任务检索与结果展示的完整流程。

现阶段，项目已由“核心功能实现”进入“工程化完善与规模化验证”阶段。团队正在围绕超大代码仓下的索引构建效率、跨语言依赖恢复、检索结果稳定性、系统异常处理及多模块协同等方面持续优化，并进一步完善面向 Coding Agent 的任务级上下文输出与接口能力。

下一阶段将重点开展百万至千万行级代码工程的规模测试和系统化评测，围绕检索准确率、响应时间、跨语言依赖覆盖情况以及 Coding Agent 接入后的端到端任务效果建立统一测试体系，并根据测试结果持续优化系统性能与工程稳定性。

第二章 背景与需求分析

2.1 行业背景与国家战略需求

2.1.1 国产软件与操作系统生态发展

随着数字中国建设和软件产业自主创新持续推进，国产操作系统、基础软件及行业应用生态不断发展，软件系统迁移、平台适配和持续演进需求日益增长。在这一过程中，大型企业长期积累了大量业务代码资产，如何快速理解已有软件工程、保障系统迁移效率并提升研发质量，成为国产软件生态建设中的重要问题。

与此同时，AI Coding 技术快速发展，大模型正在逐步应用于代码生成、代码修改和智能开发辅助。然而，在真实企业级软件环境中，代码规模通常达到百万甚至千万行级，并涉及多仓库、多模块以及 Java、ArkTS、Kotlin Multiplatform、C/C++ 等多种技术栈。受限于上下文窗口和工程理解能力，现有 AI Coding 工具难以直接掌握完整软件结构。

传统字符串搜索、LSP 和文本匹配方法难以有效恢复复杂模块之间的调用关系、跨语言依赖和隐式工程关联；依赖智能体自主探索又容易产生检索效率低、上下文冗余以及关键代码遗漏等问题。因此，面向超大规模软件工程构建代码理解与快速索引能力，成为提升国产软件研发效率、支撑 AI Coding 工程化应用的重要基础技术。

2.1.2 AI Coding 技术发展趋势

AI Coding 工具提升了代码生成效率，却没有消除大型工程中的代码定位和依赖恢复问题。随着 Coding Agent 能力不断增强，AI Coding 正逐步从代码补全、单文件生成，发展到面向完整开发任务的多文件修改、错误修复和软件迁移。

但大模型上下文容量仍然有限，无法直接容纳百万行乃至千万行工程。若按固定长度切分代码，类、接口、调用链和配置之间的联系容易被破坏；若依赖人工或智能体逐文件搜索，检索时间和模型调用成本又会随仓库规模增长。

因此，面向大型工业软件的 AI Coding 需要独立的工程级代码理解、索引和检索基础能力。

2.1.3 超大规模软件工程智能化需求

大型工业软件通常具有代码规模大、模块数量多、技术栈复杂、开发周期长和历史版本多等特点。单个开发任务所需信息往往分布于不同文件甚至不同语言模块中。

传统 LSP、字符串检索和文本匹配适合查找已知符号，但对于反射、依赖注入、配置驱动调用、跨进程接口及跨语言关系，往往难以形成完整结果。单纯依靠向量相似度同样不能解决这一问题。语义相近的代码未必具备相同接口和工程依赖，直接用于代码生成或迁移可能产生新的错误。

因此，大型软件智能研发不仅需要“找到相似代码”，还需要同时处理超大规模代码组织、多语言依赖理解、自然语言任务定位和工程上下文构建。

2.2 命题企业与业务背景

2.2.1 命题企业相关业务背景

本项目面向华为提出的“AI Coding 超大工程代码理解与快速索引”相关产业需求。

随着大型软件系统规模和复杂度持续提升，研发过程中大量新增、修改、删除、迁移和维护任务需要在百万至千万行级工程中快速确定代码位置及相关依赖。在 HarmonyOS 等大型软件生态建设过程中，多语言、多模块和跨平台工程进一步增加了代码理解难度。

因此，如何为 AI Coding 系统提供面向超大工程的代码理解和快速索引能力，成为提升大型软件研发效率和代码生成质量的重要基础问题。

2.2.2 大型软件工程与代码迁移场景

在大型工业软件中，一项看似简单的功能迁移往往涉及多个文件和模块。例如，一个业务功能可能同时关联业务入口、Service、Repository、公共数据结构、配置文件、Native 模块以及相关测试。

在 Java、ArkTS、Kotlin/KMP、C/C++ 等多语言混合工程中，相关关系还可能通过 JNI、N-API、构建配置、接口声明或框架注册建立。

因此，软件迁移并不是简单的源语言到目标语言语法转换。在真正开始代码生成之前，系统首先需要确定原功能位于何处、由哪些代码共同实现、依赖哪些接口和配置，并据此构建完整的迁移上下文。

2.3 命题需求分析

2.3.1 命题总体目标

赛题要求系统支持 Java、ArkTS、Kotlin Multiplatform、C/C++ 等语言的代码理解与检索。

代码检索准确率目标不低于 90%，平均检索时间控制在 10 秒以内。对接 AI Coding 后，转换或生成代码的功能完备度目标不低于 90%。

这些指标构成项目建设的基本约束，最终成绩需要通过统一测试集和统一运行环境进行验证。

2.3.2 命题指标拆解

从工程实现角度，可以将命题要求进一步拆分为以下几类问题：

- 百万至千万行级代码规模，对应超大规模代码仓建模和索引能力；
- Java、ArkTS、Kotlin/KMP、C/C++ 等多语言支持，对应统一代码理解与跨语言依赖分析能力；
- 较高的检索准确率，对应自然语言任务与代码之间的高召回、高精度定位问题；
- 平均检索时间限制，对应超大候选空间下的索引效率和在线查询效率问题；
- AI 代码生成和转换完整度要求，对应检索结果是否能够形成完整任务上下文的问题。

2.3.3 命题本质分析

从 AI Coding 工作流程来看，命题的核心并不是单纯建设一个更快的文本搜索系统，而是解决：

给定一个自然语言开发任务，如何从百万乃至千万行代码工程中快速找到真正相关的代码及其关键依赖，并组织成可直接供 Coding Agent 使用的工程上下文。

因此，本项目最终需要同时解决“工程如何建模”“依赖如何恢复”“代码如何检索”和“上下文如何组织”四个相互关联的问题。

2.4 典型业务场景

2.4.1 功能定位场景

开发人员可能只掌握自然语言需求，而并不知道对应代码的文件名、函数名或模块位置。

例如，当用户提出“查找登录失败后转换为 HTTP 401 的处理逻辑”时，实际实现可能位于异常处理器、鉴权模块或网关组件中，其函数名称与自然语言需求并不完全一致。

系统需要从自然语言意图出发，在大型代码仓中快速定位相关模块、文件和符号。

2.4.2 代码迁移场景

遗留系统向新语言、新框架或新平台迁移时，语法转换只是其中一部分。

类型系统、异常处理、并发模型、数值精度、内存管理和框架约定的差异，都可能导致迁移后的代码虽然能够编译，却无法保持原有行为。

因此，企业需要在复用现有业务逻辑的同时，恢复接口、调用关系、数据结构、配置和测试等必要依赖，并保留代码来源、依赖范围、修改记录和验证结果。

2.4.3 大型工程修改场景

在大型仓库中修改一个公共接口、基础类型或核心模块时，影响范围可能跨越多个业务模块甚至多个语言边界。

如果 Coding Agent 只读取目标文件，很容易遗漏调用方、实现类、配置项和测试用例。若直接读取整个工程，又会产生巨大的上下文开销。

系统需要根据当前任务识别真正受到影响的范围，为智能体提供有限但充分的工程上下文。

2.5 现有 AI Coding 工作方式分析

通用编码智能体能够读取文件、修改代码并执行测试，但在超大仓库中通常依赖多轮探索完成代码理解。

典型过程表现为：

Search / Grep → Read → 发现新的 Symbol → 再次 Search → 再次 Read
→ 逐步建立上下文。

这种方式在小型和中型工程中具有较好的灵活性，但随着代码规模、模块数量和语言数量增加，Agent 需要执行更多搜索和读取操作才能确定任务边界。

上下文不足时，关键类型、配置或调用方可能被遗漏；上下文过多时，无关内容会占用模型输入并干扰判断。因此，大型工程需要为 Coding Agent 提供更稳定的代码检索和上下文供给能力。

2.6 现有技术方案及其局限

2.6.1 关键词与全文检索

关键词、字符串和正则检索具有执行速度快、结果可解释等优点，对于已知文件名、函数名、错误码和固定字符串等场景十分有效。

但自然语言需求与代码标识符之间经常存在表达差异。当用户并不知道具体 Symbol 名称，或者不同技术栈对同一业务功能采用不同命名时，词法检索容易产生漏检。

2.6.2 LSP 与符号检索

LSP 和编译器符号索引能够提供定义、引用、类型等结构信息，对于已知 Symbol 的精确导航具有明显优势。

但其前提通常是已经知道从哪个符号开始查找。同时，不同语言和构建体系具有不同语义分析能力，跨语言桥接、运行时注册和配置驱动关系也无法仅通过单一 LSP 完整恢复。

2.6.3 向量语义检索

向量检索能够将自然语言需求与代码或代码说明映射到统一语义空间，在名称不一致的情况下仍有机会召回功能相近实现。

但语义相关并不意味着工程上真正相关。两个函数可能描述相似业务，却属于完全不同的调用路径、平台环境或模块边界。仅依赖向量相似度无法保证必要依赖完整。

2.6.4 传统 RAG

普通代码 RAG 往往将源代码按固定长度切分，再建立向量索引并直接返回 Top-K Chunk。

这种方法实现简单，但固定 Chunk 很容易切断函数、类型和模块边界，同时检索粒度与实际开发任务并不一致。“修改一个函数”和“迁移一个业务模块”显然需要不同范围的代码上下文。

2.6.5 Coding Agent 多轮探索

Coding Agent 可以通过不断执行搜索和文件读取动态探索工程，其灵活性高于静态 RAG。

但在百万至千万行代码仓中，多轮探索会产生明显的模型调用、文件读取和响应时间开销。同时，搜索方向高度依赖智能体当前上下文和中间判断，容易出现重复搜索或遗漏工程关系。

2.7 核心技术难点总结

2.7.1 超大规模代码仓建模

百万至千万行级工程包含大量文件和模块，源码、接口、配置与测试共同构成完整的软件功能。代码建模需要将这些分散的信息组织起来，建立文件、类型、函数与模块之间的对应关系。这一过程需要保留理解代码所需的结构信息，同时控制解析、存储和语义分析的时间与内存开销。工程代码持续更新时，一处修改还可能影响相关符号、依赖关系和模块信息。系统需要识别变化的影响范围，复用未变化的分析结果，使索引能够及时更新。最终形成的工程表示应支持从子系统、模块逐步定位到具体函数，适应不同粒度的开发任务。

2.7.2 跨语言跨模块依赖分析

大型工程中的一项功能经常由多个模块、不同语言的代码共同完成。模块之间的联系可能来自函数调用、类型引用和接口实现，也可能通过配置、注册和构建过程建立。例如，上层的 Java 或 ArkTS 接口可能调用 C/C++ 实现，接口声明与实际执行代码分布在不同语言的文件中。这些语言在符号命名、类型表达和绑定方式上存在差异，增加了依赖关系识别的难度。系统需要结合源码结构与工程信息，将相关接口、注册入口和具体实现连接起来。恢复的关系还应保留来源依据，支持跨模块查询、调用路径追踪和修改影响分析。

2.7.3 高精度低延迟代码检索

用户通常通过功能需求、接口描述或错误信息查找代码，表达方式与代码中的名称并不完全一致。同一项功能在不同模块中还可能使用不同命名，直接匹配关键词容易遗漏相关实现。语义检索能够发现表达不同的候选，进一步判断还需要考虑接口、模块和依赖关系是否符合当前任务。当检索范围扩大到百万至千万行工程时，读取和比较大量候选代码会增加响应时间。系统需要联合词法、符号、语义和工程关系进行

检索，逐步筛选并排序候选。关键是在较短时间内定位相关实现，同时保证完成任务所需的代码能够被召回。

2.7.4 任务级上下文构建

完成一次代码修改或迁移，通常需要同时理解目标实现及其相关依赖。例如，调整文件上传限制，除了读取大小检查逻辑，还需要了解异常类型、上层处理方式和相关测试。检索命中的函数是分析入口，系统需要进一步补充当前任务所需的接口、数据结构和配置。依赖扩展过程中，同一段源码可能被反复引用，公共模块也可能带入大量与任务无关的内容。上下文构建需要围绕任务保留完整的关键实现，合并重复源码，并补齐必要依赖。对于后续分析才需要的内容，可以继续按符号或关系补查，使 Agent 能够沿着完整的功能流程开展分析和修改。

第三章 市场现状与竞品分析

3.1 市场供给现状

与本项目相关的产品主要分为四类。第一类是代码搜索与代码导航平台，重点解决多仓库检索和符号跳转；第二类是代码重构与迁移平台，擅长按照确定规则批量修改代码；第三类是面向特定技术栈的现代化工具，提供固定升级路径；第四类是通用 AI Coding 智能体，能够根据自然语言完成代码分析和修改。

这些产品已经覆盖了代码搜索、结构分析、自动变换和智能交互等环节，但各类产品的建设重点不同。RECAST 的目标是把超大规模索引、跨语言依赖、功能候选检索和上下文组织连成一套工程能力，因此既要吸收现有产品的成熟做法，也要明确自身适用范围。

3.2 主要产品对比

产品类别	主要能力	与本项目的差异
Sourcegraph	支持跨代码托管平台和多仓库的关键词、正则、符号、提交及差异搜索。其精确代码导航基于 SCIP 索引，可提供跨仓库定义与引用关系；精确索引不可用时，可退回搜索式导航。	Sourcegraph 的优势集中在大规模代码搜索和符号导航。RECAST 还需要面向自然语言开发任务识别相关实现，并根据工程依赖组织任务级上下文。两者最直接的对标项是索引规模、检索时延和关系查询能力。
GitHub Copilot	能够利用仓库内容回答问题或执行编码任务，并与代码托管和研发协作流程结合。	其主要服务于通用开发辅助。RECAST 将超大规模代码索引、跨语言工程依赖、候选召回、上下文裁剪和来源追溯作为独立能力进行建设和评测。

产品类别	主要能力	与本项目的差异
Moderne 与 OpenRewrite	OpenRewrite 使用 Lossless Semantic Tree 表示代码，通过确定性 recipe 执行框架升级、安全修复和批量重构。Moderne 将这套能力扩展到多仓库并行执行和统一治理。	该类产品适合已知变换规则下的标准化升级。RECAST 首先需要根据自然语言任务发现相关代码，再恢复依赖并交由智能体完成具体适配。固定变换规则无法直接替代这一过程。
Amazon Q Developer	提供 Java 语言和依赖版本升级等受支持场景的自动化流程，并结合构建、差异查看和人工接受完成迁移。	其优势集中在边界明确的技术升级任务。RECAST 面向多语言大型工业软件，重点解决任务驱动的代码定位、依赖恢复和上下文供给。

3.3 竞争判断

Sourcegraph 在超大规模代码搜索与导航方面最具代表性，Moderne 与 OpenRewrite 在确定性代码变换和批量治理方面优势明显，Amazon Q Developer 已将部分技术栈升级流程产品化，通用 AI Coding 智能体则提供了灵活的自然语言操作方式。这些产品构成了 RECAST 的主要参照对象。

现有产品通常从搜索、变换或智能体操作中的某一环节切入，较少同时覆盖超大规模增量表示、跨语言工程依赖、语义与结构联合检索和任务级上下文构建。

RECAST 的差异在于将上述能力组合为面向大型工业软件和 AI Coding 的代码理解与检索基础平台。

3.4 项目差异化方向

首先，RECAST 的检索目标不只是符号和文本，而是与当前自然语言开发任务真正相关的功能实现和工程位置。系统需要识别命名和语言不同但承担相近职责的候选代码。

其次，候选排序不仅依赖语义相似度，还需要考虑调用关系、类型依赖、构建条件、目标模块兼容性和测试等工程证据，从而减少语义相关但无法用于当前任务的候选。

再次，系统交付的不只是代码片段。检索结果还应包含必要依赖、关系路径、缺失项、来源版本和证据，围绕任务组织完整实现，并支持按需补查与调用方指定的容量限制。

最后，RECAST 与通用 Coding Agent 形成能力分工。解析、建图、索引和基础检索由系统稳定执行，高成本的语义判断和代码生成由智能体在更小、更准确的候选范围内完成。

第四章 面向功能理解的代码检索框架

4.1 从功能需求到分散实现

大型工程中的功能修改始于业务描述，最终落实到分散的代码实体。用户提出“调整上传文件的大小限制，并修改超限时的异常返回”，需要找到请求入口、流式读取中的累计检查、异常定义以及上层错误处理。这些实现可能分布在不同目录中，分别以接口、服务和工具类的形式存在。目录展示了工程如何组织文件，函数名描述了局部操作，而用户关注的是这些操作如何共同完成一项功能。

从需求到实现的距离，是大型代码仓检索的核心问题。开发者通常先搜索关键词，再阅读命中文件、追踪调用、补充类型定义，逐步拼出处理流程。编程 Agent 同样需要经历这一过程：每次读取解决一个局部疑问，也可能引出新的依赖。工程越大，候选实现越多，相似名称、公共工具和跨模块调用就越容易增加查找轮次。一次检索的价值因此取决于能否把功能描述落实到可操作的实现范围，并提供理解和修改这些实现所需的关联代码。

企业代码复用与工程内功能修改共享这一需求。前者需要从参考工程中找出已有能力，理解其行为与依赖，再适配目标工程；后者需要在当前工程中定位功能，判断修改位置及影响范围。两类任务都以功能定位为起点，以可执行的代码上下文为中间结果。RECAST 将这一共同过程组织为“工程建模、任务检索、上下文构建、Agent 开发”，使同一份工程分析结果既服务于日常修改，也服务于功能复用和迁移。

4.2 离线建模与在线检索的协同架构

工程结构在多次任务之间具有复用价值，而相关代码随任务变化。系统据此将计算分为离线建模与在线检索两条路径：离线处理提取文件、声明和依赖，建立功能模块层级，并为声明、正文及职责摘要生成检索索引；在线处理解释当前需求，从已有索引中选择候选范围，沿必要依赖补充源码，整理成 Agent 可以直接使用的上下文。

离线模型包含两种相互关联的结构。模块树表达职责的包含关系，将工程逐步展开为功能域、模块及更细的实现范围。用户可以沿“上传处理”进入“请求解析”或“磁盘存储”，理解各部分的职责。依赖图表达代码实体之间的调用、类型引用和跨语

言绑定，将模块边界两侧的实现连接起来。模块树回答功能位于何处，依赖图进一步说明该功能如何运行。

全文与向量索引提供进入这两种结构的检索入口。明确的类名、方法名和异常名适合匹配声明及源码词项，自然语言功能描述适合匹配模块摘要与实现语义。命中结果统一关联到模块、文件和符号，使在线查询能够从职责描述下钻到实现，也能够从一个明确符号反查其所在模块和相关调用。

围绕这一架构，技术方案由四项相互衔接的机制组成。

关键问题	核心机制	形成的检索能力
超大工程离线建模	有界解析流水线与依赖约束的自适应分层	将分散源码组织为可逐层定位的功能范围
跨语言依赖追踪	统一符号表示与构建上下文约束的绑定恢复	连接接口、注册入口与实际实现
快速定位任务代码	模块路由、混合召回与候选按需读取	从功能描述或准确标识符进入相关实现
精简代码上下文	需求覆盖驱动的依赖扩展与源码去重	集中提供主要逻辑及其必要依赖

4.3 从功能定位到开发执行的技术主线

以调整上传限制为例，离线建模首先识别请求解析、大小校验、存储和异常处理等职责，并将其关联到实际文件与声明。在线查询利用这些职责说明定位候选模块，再结合源码中的大小检查、异常构造和调用入口检索具体函数。模块层级缩小搜索范围，声明与正文通道保留准确符号的直接命中路径，两者共同形成候选集合。

候选函数为依赖追踪提供起点。大小检查函数关联超限异常，异常又沿调用关系传递到请求入口。系统围绕需求中的“限制修改”和“异常返回”补充这条路径，并合并重复出现的声明和源码范围。最终上下文按主要实现、相关依赖和来源位置组织，使 Agent 能够沿同一条处理流程制定修改方案。

任务进入开发阶段后，工程内修改围绕现有入口和行为约束执行；复用迁移则进一步比较参考实现与目标工程的接口、类型和执行方式。编译诊断与行为测试将修改结果反馈到具体符号和分支，Agent 再围绕这些位置补充查询和修复。检索由此贯穿方案制定、代码生成与验证，使后续每一轮分析都能够复用已有的工程结构和任务上下文。

第五章 依赖约束的自适应建模与增量索引

5.1 以统一符号模型连接多语言源码

多语言工程中的代码实体具有不同表达形式。Java 通过类和方法组织行为，TypeScript 通过导出声明连接模块，C/C++ 则广泛使用函数、头文件和注册入口。检索需要把这些实体放在同一条功能路径中，因此首先要建立独立于语言语法树的结构表示。

系统将语言前端的输出归一化为文件、符号和依赖三类记录。文件描述源码所在的位置及其语言，符号描述声明的身份和范围，依赖描述两个实体之间的关系。三者通过标识关联，查询模块据此读取声明、追踪关系并定位原始正文。

结构对象	主要信息	在功能检索中的作用
文件	路径、语言、文件角色、内容哈希	确定源码位置、所属范围及变化输入
符号	声明种类、限定名称、签名、源码范围	将检索命中映射到完整类、函数或接口
依赖	源与目标、关系类型、绑定结果、来源位置	追踪调用、类型引用与跨语言实现路径

符号身份与源码位置分别维护。插入注释会改变函数行号，但函数的名称、签名和职责通常保持不变。符号键结合文件内声明标识、限定名称、签名及语言信息构造，行列范围则负责定位正文。这样的表示支持在更新位置后继续关联同一声明，也为片段去重和增量更新提供基础。

语法前端提取声明和引用，名称解析或编译器分析进一步确定引用目标。例如，同名方法需要结合参数签名及作用域区分，条件编译中的引用需要结合参与构建的代

码确定。依赖记录保存绑定依据；有多个候选时保留候选集合，缺少外部声明时保存引用及其来源位置。后续分析可以沿这些记录继续补充构建信息，把结构提取逐步推进到关系恢复。

5.2 以有界流水线控制大仓建模的内存增长

超大工程建模的内存开销来自多个同时存活的中间结果：文件正文进入解析器后生成语法树，语法树产生声明与引用，引用再转为依赖记录，最终交给存储与索引阶段。若解析持续快于写入，尚未消费的结果就会在阶段之间积累。此时，增加解析进程会扩大积压，也会使更多语法树同时驻留。

建模流水线按照扫描、解析、关系构建、批量写入和检索索引生成划分阶段。扫描器分批提交文件，解析工作进程在完成批次后释放中间对象，写入端将结构记录集中提交。流水线的背压设计同时关注队列中的任务数与字节量：任务数限制大量小文件产生的调度与对象开销，字节量限制少量大文件形成的集中内存占用。下游积压达到队列上限后暂停上游提交，消费后再恢复生产。

设阶段 i 的并发数为 p_i ，单个工作单元的平均服务速率为 μ_i ，流水线的稳定吞吐满足：

$$X \leq \min_i (p_i \mu_i). \quad (5.1)$$

这一关系决定了资源分配方式。解析成为瓶颈时，提高解析并发可以增加供给；写入成为瓶颈时，应调整提交批次和消费能力，同时限制待写入数据。解析、模型分析和向量计算分别使用受限资源池，让各阶段按自己的处理能力推进。对于大型输入，调度的重点是保持阶段之间的流量平衡，使任务持续前进且中间结果能够及时释放。

结构记录、源码正文和向量表示也按照使用方式分别存取。结构解析输出紧凑的声明与关系，正文按文件或源码范围读取，向量生成只处理需要编码的文本。后续阶段据此消费自己的输入，减少大型源码对象在各阶段之间重复传递。阶段内的批处理与阶段间的背压共同约束活动数据规模，为持续处理大仓文件提供基础。

5.3 面向功能定位的依赖约束自适应模块划分

大型工程通常按照接口、服务和存储等层次组织目录，而一次功能修改需要沿着业务行为寻找实现。以文件上传为例，请求接收、大小校验、临时文件写入和异常处理可能分布在不同目录中。直接按目录检索，会把一条完整的处理流程拆散，开发者仍需逐个阅读文件、追踪调用，才能拼出功能全貌。因此，模块建模的关键是建立从功能职责到分散实现的映射，让检索能够沿着“上传处理”这一职责找到相关代码。

为建立这一映射，本项目采用依赖约束的自适应模块划分方法，其核心是结构约束的层次化语义聚类。系统首先从目录、包结构和文件角色中提取候选代码组，汇总各组的公开接口、代表源码及组间依赖，再由 Agent 根据这些信息判断职责边界：将共同承担一项职责的候选组合并，对包含多项独立职责的范围继续拆分。每次划分形成的模块都具有功能名称、职责说明和对应的文件集合，下一轮分析则进入子模块，逐步形成从整体功能到局部实现的层级结构。

这一聚类过程结合代码之间的结构联系与功能语义。共同调用同一组实现的接口、围绕同一流程协作的类，以及跨目录密集交互的文件，为候选聚合提供依赖线索；公开接口与代表源码则帮助 Agent 判断这些联系是否服务于同一职责。聚合结果再通过文件归属约束校验，作为下一层细分的输入，使语义判断、结构校验和递归展开共同形成模块边界。

对父模块 m 的一次拆分，记其文件集合为 $F(m)$ ，子模块为 $m_1, \dots, m_k$ 。划分结果满足：

$$k \geq 2, \quad \bigcup_{i=1}^k F(m_i) = F(m), \quad F(m_i) \cap F(m_j) = \emptyset \quad (i \neq j). \quad (5.2)$$

集合覆盖保证父范围中的文件都有归属，互斥约束保证同一层的归属唯一。跨模块协作通过依赖边表达，父节点聚合子节点的文件及符号信息。这样，用户沿树浏览时能够看清责任边界，检索沿图扩展时又能连接这些边界两侧的代码。

层级按分支自适应展开。职责集中的小范围直接成为叶模块，包含多个独立功能的大范围继续细化；文件与类、函数通过叶模块进入具体实现。一个小工程可以采用浅层结构，大型工程则在复杂分支中逐步形成子系统、模块和子模块。每层划分都服务于缩小功能范围，层级深度随工程自身的复杂程度变化。

父节点确定子范围后，各子模块可以独立分析。模型每次面对局部候选及必要源码，已经完成的分支可以直接保存和复用。局部输入降低单次职责分析的规模，分支调度则将整仓分析转化为一组范围明确的建模任务。最终模块树同时提供功能名称、职责总结和源码入口，为在线检索建立从整体到局部的导航结构。

5.4 以构建上下文恢复跨语言调用路径

功能跨越语言边界时，调用联系往往隐藏在注册与导出机制中。Java 声明可能通过 JNI 连接 Native 函数，ArkTS 接口可能经由 N-API 导出连接 C/C++ 实现，KMP 公共声明则通过源集关系连接平台实现。上层名称与底层函数名可以不同，连接两者的关键是语言边界处的绑定信息。

跨语言绑定恢复以“声明、绑定入口、实际实现”为基本路径。对于注册式 Native 调用，先从上层声明提取方法名和签名，再查找匹配的注册项，最后解析注册项引用

的底层函数。依赖图分别保存声明到注册项、注册项到实现的连接，并关联各自的源码位置。查询从上层接口出发，便可沿注册信息进入底层正文。

绑定判断还需要明确代码参与构建的范围。Java 的类路径决定可引用的类型，C/C++ 的宏和头文件搜索路径影响实际声明，KMP 的源集关系决定公共接口对应的平台实现。因此，绑定过程结合导出名、签名、库范围及构建条件筛选目标。同一注册名出现在不同库或不同签名下时，分别保留其适用范围，使调用路径与实际构建条件对应。

这一机制将跨语言检索从名称关联推进到实现追踪。Agent 迁移某个上层接口时，可以同时获得注册映射、底层函数以及必要的类型与配置；调整底层行为时，也可以沿绑定关系回查受影响的上层入口。统一符号模型负责容纳这些实体，绑定恢复负责建立边界连接，两者共同构成跨语言功能分析的基础。

5.5 按变化影响范围更新模型与索引

离线模型需要随源码演进持续更新。正文变化、公开接口变化和构建配置变化影响的对象不同：修改函数内部逻辑主要改变实现片段，修改签名会影响引用该声明的关系，调整注册表则可能改变跨语言调用目标。增量更新据此按变化类型传播，而文件内容哈希用于识别最初发生变化的输入。

更新首先定位变化文件，复用其余文件的结构结果，再比较变化前后的声明及关系。系统通过源文件、符号与派生记录之间的反向映射找到受影响对象，重新生成对应的源码表示、绑定关系和职责摘要。模块说明沿包含关系更新到受影响的祖先，向量表示依据实际编码文本及编码模型配置复用。

变化类型	主要影响范围	更新操作
实现正文变化	源码片段、局部引用、所属模块说明	重新解析变化正文，更新相关关系及检索表示
公开声明变化	声明记录、引用方、接口依赖	更新符号并重新绑定受影响引用
构建或注册变化	依赖构建条件的语言内及跨语言关系	重新确定候选范围与绑定目标
职责划分变化	模块归属、祖先摘要、模块检索表示	更新相关分支并汇总上层职责

设初始变化集合为 A_0 ，由对象集合 A 派生的受影响记录为 $\text{Derived}(A)$ ，需要同步更新的模块祖先为 $\text{Ancestors}(A)$ ，影响传播可表示为：

$$A_{t+1} = A_t \cup \text{Derived}(A_t) \cup \text{Ancestors}(A_t). \quad (5.3)$$

传播规则按变化类型选择关系，直至受影响集合不再扩大。一次上传限制逻辑的修改由此可以更新对应函数及职责摘要；若同时改变异常接口，则进一步更新其引用关系。增量机制复用稳定部分，把重复建模的工作集中到变化所影响的结构与语义范围，使离线模型能够持续服务后续任务。

第六章 任务驱动的分层检索与上下文构建

6.1 以职责层级缩小搜索范围

用户对代码的需求具有不同粒度。修改一处校验逻辑时，主要结果是函数及其直接依赖；理解一个接口时，需要类、方法约定和实现关系；迁移一项功能时，则需要模块范围及其内部协作结构。系统将函数、类、模块和子系统作为可选择的结果粒度，并提供自动模式，让查询范围与当前任务相匹配。

显式粒度决定主要结果的组织方式，必要依赖则按实际声明补充。例如，选择函数粒度定位上传限制，主要结果可以是执行检查的方法，同时附带异常类型和调用入口；选择模块粒度迁移上传能力，则以职责范围组织文件，再展开关键接口及实现。粒度选择使用已有的结构和模块索引，因此可以在同一工程模型上切换观察范围。

分层检索先比较需求与模块职责摘要，将查询引向相关分支，再在这些分支中匹配类和函数。对于“上传限制”，上层职责提供上传处理范围，下层模块进一步区分请求解析、流式读取和异常处理。逐层缩小范围后，细粒度检索集中比较更相关的代码。

同时，全局声明和正文检索提供直接命中路径。用户给出准确方法名或异常名时，该通道可以直接定位声明，再关联到所属模块。模块路由与全局通道共同提供候选，使功能描述和准确标识符都能成为有效入口。

多项要求组成的任务还需要保持各部分的搜索线索。“调整上传限制并修改异常返回”包含校验逻辑与错误处理两个关注点，检索设计据此分别寻找相关实现，再利用调用关系连接结果。这样的任务分解为后续上下文构建提供覆盖目标，使选择过程能够识别尚需补充的功能环节。

6.2 融合声明、正文与模块语义的混合召回

同一项功能在不同表示中暴露出不同线索。声明包含方法名、参数和返回类型，正文包含分支条件与实际调用，模块摘要则将一组实现概括为功能职责。准确的 `parseRequest` 或 `FileUploadBase` 适合通过声明定位，“上传大小限制”则可以通过

过职责摘要和正文语义找到对应实现。混合召回将这些表示作为互补的查询通道。

系统分别围绕声明、源码正文和模块摘要组织全文与向量检索。全文匹配保留标识符及关键源码词项，向量匹配连接自然语言需求与不同命名下的实现语义。各通道先返回候选，再通过统一的模块或符号标识关联到实际代码实体。

不同通道的分值具有不同尺度，因此融合采用基于名次的 RRF 方法。对候选 d ，融合得分为：

$$\text{RRF}(d) = \sum_{j: d \in L_j} \frac{w_j}{k_0 + \text{rank}_j(d)}. \quad (6.1)$$

其中， L_j 为第 j 个通道的结果列表， w_j 为通道权重， k_0 为平滑常数， $\text{rank}_j(d)$ 为候选在该通道中的名次。一个实现在多个通道中获得靠前名次时，会积累更高的融合得分；只在某个通道中具有明确线索的候选，也可以通过该通道进入后续筛选。

正文命中进一步映射到所属声明。若片段位于函数中部，系统通过源码范围找到完整函数，使候选携带方法签名及正文边界。同一符号被多个通道命中时合并记录，模块候选则关联其文件范围与代表符号。融合后的结果因此能够按函数、类或模块继续处理。

精排设计围绕任务适用性展开。工程范围、语言和目标接口先形成筛选条件，有限候选再依据具体行为、接口约定及边界依赖进行比较。模型参与判断时读取这些候选的声明和必要正文，重点解释各自满足哪部分需求。结构筛选与语义判断由此形成前后衔接：前者压缩比较范围，后者区分功能接近但使用条件不同的实现。

6.3 通过候选按需读取降低在线开销

在大型代码仓中，查询开销既来自检索计算，也来自结果读取。若每个召回通道都提前加载大量源码，随后大部分候选又在融合时被淘汰，数据库传输和正文构建就会消耗在未被使用的内容上。同样，为了查找少量符号而加载整仓结构，也会把局部查询扩大为全局数据处理。

在线查询采用先定位、后取数的处理顺序。召回阶段优先取得候选标识及排序所需信息，工程与模块范围尽早参与过滤；候选融合后，再按标识读取参与精排的声明与正文。依赖补充通过局部邻接查询获取相关目标，多个目标集中读取，已取得的符号与源码在同一查询中复用。

这一方式使读取范围与当前候选集合及相关依赖相联系。以上传限制为例，模块检索先返回请求解析及校验相关的范围，符号检索返回候选方法；进入正文比较后，再读取这些方法及关联异常。存储接口承担按符号、关系和源码范围取数的工作，查询服务可以逐步取得任务所需内容。

不同召回通道还可以并行执行，相同编码输入的查询向量在通道间复用。并行召

回缩短独立搜索之间的等待，延迟读取减少融合前的无效取数，局部依赖查询则控制命中后的数据扩展。三者分别作用于搜索、候选处理与依赖补充，使在线优化贯穿整个查询过程。

6.4 沿任务相关依赖补全实现路径

首次命中的函数为理解功能提供了起点，完整修改通常还需要它使用的类型、调用的实现以及影响的入口。对于上传限制，累计大小检查决定何时触发异常，异常类型定义可供上层读取的信息，入口处理则决定如何向调用方返回错误。这些代码与任务有关的原因来自行为链上的分工。

为连接这些实现，方案采用需求覆盖驱动的依赖子图抽取，以检索命中的符号为起点，由任务类型决定扩展方向。修改函数内部逻辑时，优先查看被调用实现和必要类型；调整公共接口时，进一步查看实现类及调用方；迁移跨语言功能时，优先连接声明、注册入口与平台实现。每个新增实体都保留与任务入口的关系，逐步形成包含主要行为及必要依赖的局部子图。

遍历过程中，已访问符号按标识去重，共享依赖只读取一次。遇到公共工具或调用方较多的节点时，继续展开的依据是该分支是否参与当前关注的行为。例如，异常返回任务需要进入处理该异常的上层分支，存储路径下的其他辅助操作则由具体修改需要决定。任务相关性由此约束图遍历的方向和范围。

为组织包含多个关注点的需求，方案将需求项表示为集合 U ，每项权重为 $w(u)$ 。候选片段 e 所涉及的需求项记为 $\text{Cov}(e)$ ，已选集合 C 的覆盖得分为：

$$F(C) = \sum_{u \in \bigcup_{e \in C} \text{Cov}(e)} w(u). \quad (6.2)$$

入口、主要分支、异常行为、接口依赖和相关测试都可以成为需求项。其与代码的对应关系结合声明、调用关系及实际正文判断。对于上传限制任务，检查函数覆盖触发条件，异常定义覆盖错误信息，入口分支覆盖返回行为；三者共同形成任务所需的实现路径。

覆盖表示进一步指导子图扩展时的候选选择。已经选入大小检查逻辑后，另一段类似检查带来的新增覆盖可能较少，而关联异常和处理入口能够补齐剩余环节。每轮扩展据此优先补充尚未覆盖的行为或必要依赖，逐步连接任务相关的实现路径；随后将子图中的符号映射回源码范围，交由上下文构建阶段合并重叠内容。

6.5 以必要覆盖和冗余消除构建精简上下文

上下文构建需要同时保留功能链的必要信息，并减少重复源码。函数可能同时被声明检索和正文检索命中，类级结果也可能包含已选方法；多个入口还可能引用同一个异常或接口。如果直接拼接这些结果，重复内容会增加阅读负担，主要实现与依赖之间的关系也容易被淹没。

上下文整理模块首先将候选统一为可定位的源码范围。相同来源的重复片段合并，同一文件内相互包含或重叠的范围进行整理，方法正文保留其声明边界。类级与方法级结果根据任务所需粒度协调，既保留理解行为所需的接口信息，也让主要实现集中呈现。

在覆盖得分的基础上，以源码重复程度衡量冗余，候选集合的比较得分可表示为：

$$J(C) = F(C) - \lambda \text{Redundancy}(C). \quad (6.3)$$

其中， $\text{Redundancy}(C)$ 衡量重复源码范围， λ 调节冗余惩罚。覆盖集合去除同一需求项的重复计分，源码合并进一步去除文本层面的重复。对于覆盖相同且重复程度相当的集合，优先选择源码量更少的结果，使主要实现和必要依赖集中呈现。

上下文整理优先保留核心实现与必要依赖，合并重复源码，并复用已交付内容。实现正文与辅助声明分别组织，已被完整实现包含的声明和片段不重复交付。容量限制仅在调用方显式设置时生效，省略内容保留相应说明。最后检查各片段的来源和关联关系，按主要实现、必要依赖及相关测试组织输出。

例如，调整上传限制的上下文可从请求入口开始，接着呈现流式读取中的大小检查、超限异常定义及入口处理分支。若检查函数在多个召回通道出现，正文只保留一份；若异常定义被多个方法引用，则集中提供定义并关联使用位置。Agent 得到的是围绕修改目标组织的实现材料，可以据此判断应修改的条件、受影响的返回行为以及需要验证的边界。

后续补查复用已经交付的片段标识与内容哈希，优先返回新增实现或变化内容。模型输入窗口作为交付时的容量约束，主要通过去重、局部展开和分轮补充处理。上下文构建的目标始终是覆盖任务所需行为与依赖，使每次交付都推进对当前修改问题的理解。

第七章 基于代码上下文的 Agent 开发协同

7.1 将任务检索接入 Agent 的分析与修改流程

Agent 制定修改方案需要知道实现入口、相关行为及依赖关系。工程模型负责提供稳定的结构基础，任务检索负责从中选择当前需要的代码，上下文则把这些结果组织为可阅读、可定位的输入。开发流程从需求检索开始，Agent 围绕返回的实现制定方案，再根据分析中出现的具体疑问补充源码。

一次补查可以指向某个符号、调用关系或源码范围。例如，Agent 已经读到上传大小检查，却需要确认上层如何捕获异常，便沿调用关系读取相关处理分支；若需要调整接口，则继续查询实现类和调用方。每次补查都承接已有分析，把新的问题落实到工程中的具体对象。

浏览器工作台通过模块树展示离线建模后的职责层级，用户可以选择工程范围和结果粒度，查看模块总结并进入源码。HTTP、MCP 与 IDE 接入共享检索服务，使交互界面和编程 Agent 使用相同的结构结果与源码来源。用户确定关注范围后，Agent 可以在该范围内检索、分析并形成修改。

这一协同方式把可复用的工程理解前置到离线阶段，将与任务有关的判断保留在开发过程中。首次检索提供主要实现，后续查询解决局部问题，代码改动与验证反馈又为下一轮查询提供更准确的符号和位置。检索因此成为 Agent 分析过程中的持续能力。

7.2 从参考实现到目标工程的适配规划

功能复用在定位代码之后，还需要建立参考行为与目标工程之间的对应关系。参考实现的入口、数据类型、异常方式和外部依赖共同决定了它如何工作，目标工程则提供新的接口约定和运行环境。例如，同步返回结果的参考方法迁移到异步接口时，需要同时调整调用方式、返回类型及异常传播路径。

迁移流程以参考上下文和目标需求说明作为输入。Analyzer 先梳理功能行为，再

建立来源符号与目标符号之间的名称和类型映射，确定外部依赖的替代方式，并将改动组织为具有前后关系的步骤。接口定义先于依赖该接口的实现，公共类型先于使用这些类型的方法，调用方调整则跟随相应接口变化。

这样的计划把分散的文件修改连接到同一个行为目标。对于上传功能迁移，计划可以依次明确目标请求入口、流式读取方式、大小检查位置、异常类型和错误返回约定。每一步都对应参考上下文中的相关实现，并说明目标工程中需要形成的代码关系。

Translator 按计划完成多文件修改，并在需要时补充读取相关符号与依赖。编译诊断再将类型不匹配、缺失引用和调用签名问题反馈到具体步骤，推动接口映射与实现同步调整。规划、生成和编译反馈由此围绕共同的功能上下文协作，保持跨文件改动之间的联系。

7.3 以行为验证连接修改结果与任务需求

任务需求定义了修改后应呈现的行为，验证用例据此组织正常路径、边界条件和异常路径。以调整上传限制为例，需要分别检查允许范围内的上传、触及限制边界的输入以及超限后的错误返回。对于迁移任务，同一组输入还可以分别作用于参考实现和目标实现，比较输出及可观察副作用。

编译检查负责检验生成代码的语法、类型与工程依赖，行为测试进一步检查修改后的执行结果。目标接口和测试范围由需求说明确定，测试输出保留输入、预期行为与实际结果，使失败能够关联到具体的功能环节。

验证反馈反过来指导检索。若编译发现异常类型引用错误，Agent 补查目标类型及相关接口；若超限测试返回了错误结果，则回到异常构造和入口处理分支；若正常上传行为发生变化，则检查主要读取流程及受改动影响的调用。失败信息由此缩小下一轮分析范围，已有上下文继续复用。

从完整任务看，离线模型先提供职责层级和调用关系，任务检索据此定位上传处理链，上下文构建集中提供检查与异常相关代码，Agent 再完成修改或迁移，并通过编译与行为用例验证结果。最终代码差异连同参考实现、修改计划和验证结果交给开发者审阅，使需求、实现与验证能够沿同一项功能对应起来。

第八章 项目进展与规划

8.1 项目研发起点

项目起点是潍柴公司在大型软件项目中复用历史模块的实际需求。企业已积累大量代码与业务实现，但在承接新需求时，开发人员仍需从庞大的历史工程中辨认哪些模块具备相近功能、能否直接复用，以及迁移到当前项目需要调整哪些接口和依赖。如果已有实现难以被准确找到，团队就可能重复开发相同功能；通过复制代码完成复用，又容易增加代码冗余和后续维护成本。

围绕这一需求，团队将研发重点确定为从历史工程中精准查找可复用模块，并辅助完成目标项目中的适配与迁移。在需求梳理和原型开发过程中，团队搭建了代码检索、复用与迁移 workflow Demo，将需求输入、历史实现查找、候选选择和目标工程适配串联起来，为验证企业场景中的使用流程提供了基础。

这项前期工作明确了项目的应用目标：通过复用已有代码减少重复开发工作量，同时避免无约束复制带来的代码冗余。后续对大规模索引、模块理解和跨语言依赖的优化，均围绕“能否找到合适的历史实现，以及能否将其正确用于目标工程”展开。

8.2 项目开发时间线

赛题发布前，团队已围绕潍柴公司的需求开展研发，形成代码检索、复用与迁移 workflow Demo。该阶段重点是打通业务流程，使开发者能够从功能需求出发，查找历史工程中的候选实现，再进入目标项目的适配与迁移过程。原型为后续技术优化提供了已有代码、服务接口和可操作的验证流程。

2026 年 8 月底赛题发布后，团队对照赛题要求重新分析原有方案的能力。随着目标扩展到百万至千万行工程、多语言依赖和受限时延下的精准检索，研发重点进一步转向三个问题：如何控制大型工程建库的时间与内存，如何从目录和符号组织中识别可复用的功能模块，以及如何补齐迁移所需依赖并减少无关代码输入。

从 8 月底至今，团队在原有 Demo 基础上持续优化结构解析、模块分层、混合检索和任务上下文整理，并补充并行处理、增量复用、执行恢复及行为测试等能力。技术改进通过真实源码建库、模块与检索实验及功能回归逐步验证，形成当前系统。项

目因此由企业需求驱动的工作流原型，发展为能够支撑更大工程和更复杂复用任务的代码理解、检索与开发辅助系统。

8.3 版本迭代过程

初始 Demo 以潍柴的代码复用需求为中心，优先连接历史代码检索与目标项目迁移。该版本重点验证用户能否沿完整流程找到候选实现并开展适配，将原本分散的代码查找、候选判断和迁移操作组织为连续工作流。

赛题发布后的迭代首先强化大型工程的表示与定位能力。系统在原有流程中加入更细致的文件、符号和依赖记录，利用模块树表达功能职责，并结合声明、源码正文和模块摘要进行多粒度检索。由此，候选选择不仅依靠名称或局部文本相似性，还能够结合模块范围与依赖关系判断历史实现是否适合当前任务。

随着上述能力接入，系统继续优化执行效率与修改可靠性。建库流程限制解析进程和在途源码量，模块分析支持独立分支并行与已完成决策复用；在线查询共享向量编码结果、并行召回，并通过完整实现读取和源码去重组织任务上下文。迁移流程则进一步完善适配规划、编译反馈、行为测试和回滚，使系统既能辅助找到可复用模块，也能检查适配结果是否满足目标项目要求。

8.4 当前系统完成情况

系统已完成代码仓登记、源码快照、分批并行解析、结构索引存储与分析版本管理。解析任务同时受到工作进程数和在途源码字节数限制，已完成结果分批保存；兼容的解析记录可以在后续构建中复用。Java、TypeScript、JavaScript、Python、C#、Go 和 Rust 已接入结构分析，C/C++、Kotlin 及 ArkTS 的兼容语法也已具备结构解析入口。

工程理解方面，系统已实现模块职责提案、文件归属校验、自适应层级和按需细化。同级分支可以并行分析，校验后的决策持久保存，恢复执行时复用符合输入条件的结果。依赖分析使用统一关系记录，并已接入明确 JNI 导出、N-API 注册链和同模块 KMP 声明与实现的匹配规则；无法确定的目标保留歧义或未解析说明。

检索方面，系统已实现声明、正文和模块摘要的混合召回，支持工程范围、分析版本和结果粒度选择。独立召回通道并行执行，同一查询的向量编码结果共享；返回源码按需求覆盖、重复范围和输入长度筛选，并记录文件位置、依赖关系以及省略原因。HTTP、MCP、浏览器工作台和 VS Code 扩展均已具备相应的查询入口。

开发协同方面，系统已实现适配规划、多文件生成、编译反馈和固定行为测试。验收脚本由运行环境配置，生成过程不能修改受保护的验收文件；测试前后核对源码快照，后续写入会使旧验证结果失效。任务状态、文件差异和恢复记录持久保存，支

持中断后继续执行及回滚。

8.5 当前产品形态

当前产品由本地分析与检索服务、浏览器工作台、VS Code 扩展及 Agent 工具接口组成。用户可以登记工程、查看建库与模块分析进度，在模块树中选择范围，再输入自然语言需求并查看相关源码、符号和依赖。函数、类、模块与子系统结果分别展示，解析失败和待细化状态保留具体原因，便于判断当前工程信息是否充分。

对于自动化开发任务，Agent 可通过 MCP 或 HTTP 获取任务上下文，并按需补查符号、依赖与源码片段。工作区修改服务已经能够执行多文件适配、编译和行为测试，保存计划、差异与运行结果。当前查询功能已形成可操作的演示流程，迁移任务的完整页面审阅仍在继续完善。

8.6 阶段性测试环境

本阶段围绕大规模工程离线建模、平均检索响应时间和依赖分析开展验证。离线建模采用 VS Code 真实源码，模块职责分析使用 DeepSeek；检索测试采用 Commons FileUpload 的 Java 源码与 TypeScript 工程；跨语言依赖测试采用 JNI、N-API 和 KMP 绑定样例。

检索实验在 Intel Core i7-14650HX、Node.js 24.15.0 和本地 SeekDB 环境下执行，语义编码采用 384 维 multilingual E5-small。实验使用固定源码快照与已构建索引，各项任务预热一次后，顺序发起正式计时请求。

8.7 阶段性验证结果

8.7.1 大规模工程离线建模

系统对 VS Code 工程完成源码扫描、结构解析和索引构建，从 7,708 个文件中提取 287,471 个代码符号。工程共计 2,356,695 物理行，包含源码、测试、生成代码和配置等文件。

在结构索引基础上，系统对主工程开展职责分析与模块分层，生成 66 个模块和 6 个子系统，最多形成 4 层结构。主工程的 7,702 个文件均已确定层级归属，父子节点的文件、类型和方法统计经遍历核对保持一致。

建模结果已接入工作台。用户可以从“启动引导”子系统逐级进入“主进程启动”模块，继续定位到 `main.ts` 中的函数，实现从工程职责到具体源码的连续导航。当前模块树保留待细化分支，支持后续按需展开。

表 8.1: VS Code 工程离线建模结果

验证项目	实测结果
工程规模	7,708 个文件，2,356,695 物理行
代码符号	287,471 个
模块结构	72 个节点：66 个模块、6 个子系统
层级深度	最多 4 层
主工程文件归属	7,702 / 7,702 个文件

8.7.2 平均检索响应时间

检索实验选取 Java 与 TypeScript 工程中的 6 类功能需求，共构成 12 项函数级任务。每项任务重复执行 5 次，共记录 60 次请求。测试采用声明、源码和模块摘要的并行混合召回，结合依赖扩展构建任务上下文，调用时未设置 Token、文件数或源码行数上限。

平均响应时间为 **136 毫秒**。计时从客户端发起请求开始，到接收并解析完整响应结束，包含在线检索、源码读取和上下文构建。该结果对应上述 Java 与 TypeScript 函数级任务集，索引构建在计时前完成。

表 8.2: 函数级检索响应时间

测量项目	实测结果
任务规模	12 项任务，覆盖 Java 与 TypeScript
测量次数	每项 5 次，共 60 次请求
平均响应时间	136 毫秒

上下文功能回归覆盖完整函数读取、长函数分段拼接、循环依赖处理和源码去重，并检查辅助声明与实现正文的分别交付。系统默认保留已选内容，支持调用方显式设置容量上限及复用已交付源码；回归检查输出与原始源码的一致性。

8.7.3 依赖分析

在 VS Code 工程中，系统提取了 **117,420 条导入、导出及工程引用关系**，用于文件与模块之间的依赖查询。跨语言验证进一步检查源码调用、注册信息与目标实现之间的连接，覆盖 JNI、N-API 和 KMP 三类典型绑定。

各类样例均检查源符号与目标符号是否正确对应。修改 N-API 注册信息后，增

表 8.3: 跨语言绑定样例验证

绑定类型	分析路径	验证结果
JNI	Java 原生方法声明与 C/C++ 导出函数	依据导出名及参数类型描述符，定位对应实现
N-API	ArkTS 调用、模块注册与原生回调	沿显式注册链，恢复调用到原生实现的连接
KMP	同模块的 <code>expect</code> 声明与 <code>actual</code> 实现	连接预期声明与平台实现，保留多实现歧义

量分析能够更新相关调用关系，结果与相同源码的全量构建一致。缺少注册、模块不匹配或存在多个候选目标时，系统记录未解析或歧义状态。

8.8 经济效益分析

8.8.1 效益来源与测算方法

命题背景指出，超大工程中的解析与索引效率瓶颈、多语言调用链路还原困难，会使依赖链路断裂、检索噪音增大、代码定位不准，进而导致 AI 代码生成准确率低、需求理解偏差和批量改造效率低下。本节不重复定性判断，而是把这组损失逐项折算为可复算的量化指标。

效益的传导路径为：技术能力改变研发行为，研发行为改变可测量指标，可测量指标再按人日、交付周期和模型调用量折算为财务口径的收益。测算中的数值分为三类并分别标注：外部权威基准用于刻画现状损失的严重程度，平台实测值用于刻画系统当前达到的能力，待企业标定值用于表示只有试点才能确定的系数。第三类参数在表 8.4 中逐项列出，并说明标定方法，不预先假定有利取值。

上下文容量与工程规模之间的数量级失配，是全部效益的来源。按命题给出的换算关系，256K 上下文约覆盖 2 万行代码；命题描述的单模块规模为十万行，已达单次上下文容量的 5 倍，全工程百万至千万行则为单次上下文容量的 50 至 500 倍。若二者处于同一数量级，”将工程完整纳入提示词”即为可行方案，工程级检索便无独立价值。正因为存在 50 至 500 倍的失配，”从工程中筛选出当前任务真正需要的那一部分”才成为每个开发任务都必须支付的固定成本。RECAST 的作用是把这项成本从每个任务的重复支付，转为建库时的一次性支付。

表 8.4: 经济效益测算参数与数据来源

参数	含义	取值来源	状态
N	参与大型工程维护、复用与迁移的研发人数	企业实际	待标定
D	人均年有效研发工时	220 人日	通用假设
α	代码定位、阅读与依赖梳理占研发工时比例	外部实证研究	有基准
β	平台可压缩该环节的比例	企业试点	待标定
p	新增功能与既有实现存在重叠的比例	外部实证研究	有基准
γ, ρ	可识别复用比例、复用后残余适配比例	企业试点	待标定
T	年跨文件 AI 编码任务数	企业实际	待标定
C_0, C_1	有无平台时单任务模型调用成本	试点 A/B 实测	待标定
M, L, τ	年迁移模块数、单模块投入、周期压缩比	企业实际与试点	待标定

8.8.2 降低代码定位与理解成本

大型软件系统的维护投入并不主要用于修复缺陷。对应用软件维护构成的经典实测显示，完善性维护约占 56%，适应性维护约占 23%，纠错性维护仅约占 21%^[1]。前两类合计约 79%，都以读懂既有代码、确认依赖范围为前提。另一组来自搜索引擎企业内部的实证研究进一步表明，代码检索是开发者的高频日常活动，其动因之一正是寻找可复用的既有实现；十年后的复现研究发现这一需求依然存在^[2]。因此，代码定位与理解是维护成本的主要构成，而不是附带开销。

RECAST 对这一环节的作用可以直接用同一语料、同一查询的前后对照衡量。在 VS Code 主工程（7,702 个文件）上，构建”定位相关实现并补齐依赖上下文”的完整任务查询，单次耗时由约 19 至 20 秒降至 4.955 秒，返回 10 条结果、17 条源码证据，覆盖 4 个文件、260 行源码，所需方法未被截断，上下文为 7,813 个 token。在同一批 Java 与 TypeScript 函数级任务上，12 项任务各重复 5 次、共 60 次请求，包含检索、源码读取与上下文构建的平均响应时间为 136 毫秒；改用并行混合召回后由 163 毫秒降至 136 毫秒，且 60 对输出内容逐次核对一致。命题要求平均检索时间控制在 10 秒以内，实测结果保留了约两个数量级的余量。

据此建立工时测算：

$$\Delta H_1 = N \times D \times \alpha \times \beta$$

(8.1)

其中 N 为参与大型工程维护、复用与迁移的研发人数， D 为人均年有效研发工时， α 为代码定位、阅读与依赖梳理占研发工时的比例， β 为平台可压缩该环节的比例。 α 与 β 均不预先设定有利取值： α 以公开实证研究为基础， β 由企业试点的同任务对照实验标定。表 8.5 给出每千名研发人员的敏感性区间，价值换算采用命题企业 2025 年年度报告口径（研发费用 1,923 亿元除以研发员工约 11.4 万人，得到全口径研发投入约 168.7 万元每人年）^[3]。

表 8.5: 每千名研发人员的定位与理解工时节省测算

α 取值	$\beta = 10\%$	$\beta = 20\%$	$\beta = 30\%$
20%	4,400 人日	8,800 人日	13,200 人日
25%	5,500 人日	11,000 人日	16,500 人日
30%	6,600 人日	13,200 人日	19,800 人日

按全口径研发投入折算，上表对应每千名研发人员每年约 3,374 万元至 15,182 万元。需要说明的是，全口径包含设备、云资源与管理分摊，是价值换算的上限口径；若改按人力薪酬口径折算，同一工时节省对应的金额显著更低。正式测算将以人力薪酬口径作为结论，全口径数据仅用于说明研发工时的全部机会成本。

8.8.3 提升代码资产复用率，减少重复开发

企业长期积累的业务实现分散在不同仓库、模块和技术栈中，已有功能难以被及时发现。公开生态的实测显示，代码重复是软件工程的普遍现象：对 GitHub 全量代码的分析发现约 70% 的代码存在重复^[4]。更值得关注的是重复的后果而非重复本身——多项实证研究的一致结论是，克隆代码未必更易出错，真正的风险集中在“本应同步却已经分叉”的克隆^[5]。这一结论恰好指向本项目的价值：企业缺少的不是“禁止拷贝”的规范，而是**全局资产检索与同源追踪能力**。

RECAST 的跨仓库复用检索实验直接检验了这一能力。在由 13 个无关仓库与 3 个功能等价实现（C#、Python、TypeScript）构成的混合语料中，对 102 个目标类进行功能等价检索：一阶段召回率为 97.06%，最终 Top-4 召回率为 94.12%，Top-4 命中率为 97.06%；后续翻译与编译验证的编译通过率为 91.18%。该结果表明，系统能够在包含大量干扰仓库的真实混合语料中，以可接受的候选数量定位到功能等价的既有实现，直接对应命题对检索准确率不低于 90% 的要求。

复用带来的收益已有工业实证：美国国防部公开文件汇总的案例显示，NASA 软件工程实验室报告研发成本下降 55%，Boeing 报告返工下降 31%、投资回报率 7.75:1^[6]。这些数字来自不同组织与不同项目类型，只能作为量级参考，不能直接套用于本项目的预测。工时测算因此采用如下形式：

$$\Delta H_2 = F \times p \times \gamma \times W \times (1 - \rho) \quad (8.2)$$

其中 F 为年新增或改造的功能模块数， p 为与既有实现存在功能重叠的比例， γ 为其中可被检索识别并转为复用的比例， W 为单功能模块的平均实现投入， ρ 为复用后仍需适配的残余工作量比例。 p 不直接采用公开生态的重复率，而是在目标代码库上实测克隆群数量、同源分叉比例与重复行占比后确定； γ 与 ρ 由试点标定。

8.8.4 降低 AI Coding 的上下文与探索成本

把完整工程纳入模型上下文并不可行，且“窗口更大”并不等于“更会使用上下文”。多项评测给出了一致的结论：关键信息位于上下文中部时，多文档问答准确率下降约 20 个百分点，并且扩展上下文版本并不优于其短上下文同门模型^[7]；模型的有效上下文长度普遍远低于其宣称长度^[8]；在移除字面匹配、必须依赖语义关联的设定下，多数模型在 32K 长度时跌至其短上下文基线的 50% 以下^[9]。真实代码检索正是需求描述与实现代码之间没有字面重叠的语义匹配，退化只会更严重。

更关键的是，在大型代码库中，通用 AI 助手的净效应可能是负的。一项随机对照实验以 16 名资深开发者为对象，让他们在自己长期维护、熟悉的大型开源仓库中完成 246 个真实任务：使用 AI 工具后任务完成时间增加 19%，而同一批开发者事前预测会快 24%、事后仍主观认为快了约 20%，预期与现实之间存在约 40 个百分点的

落差^[10]。该实验的工作场景与大型工业软件迁移高度相似。这说明 AI 提效并非自动发生，工程上下文的供给能力是决定其正负的关键变量。

RECAST 在成本侧的处理方式与逐任务探索不同。系统按 token、文件数和源码行数三个维度对最终返回文本施加约束，用 cl100k_base 对实际导出文本重新计量，按仓库、版本、路径与内容哈希识别重复片段并合并相互包含的源码范围。大模型只参与离线的模块职责分析：在 7,702 个文件的主工程上仅消耗 24 次模型请求，即形成 66 个模块与 6 个子系统；在线检索使用本地索引与 384 维语义编码模型，不调用大模型。由此，平台的模型成本集中在建库阶段并可被后续全部任务分摊，在线查询的边际模型成本接近于零。作为对照，开放式智能体方案在仓库级修复任务上的公开成本约为每实例 0.34 美元^[11]，而图引导的定位方案报告成本降低约 86%^[12]，说明结构化定位对成本的影响可达数量级。

Agent 上下文成本的测算形式为：

$$\Delta C_3 = T \times (C_0 - C_1) + \Delta H_3 \quad (8.3)$$

其中 T 为年需跨文件理解的 AI 编码任务数， C_0 与 C_1 分别为未接入与接入平台时单任务的模型调用与检索成本， ΔH_3 为因依赖遗漏导致的返工与人工补查工时。 C_0 与 C_1 由试点 A/B 对照的 token 计量与模型定价直接换算，不依赖估算。

上述成本结构的差异可以用公开的同基准对照说明。在同一条仓库级修复基准上，开放式智能体方案的单实例成本约为 4.01 美元，而先结构化定位、再一次性生成的三阶段流水线约为 0.34 美元^{[11][17]}，两者相差约一个数量级。定位环节本身并不廉价：对定位类智能体轨迹的统计显示，单条轨迹的 token 消耗中位数约为 2.86 万^[18]。这说明“找代码”的开销足以影响整个任务的经济性；把它从每个任务中剥离出来、交由确定性索引完成，改变的并非速度，而是成本结构。

在组织层面，提效同样不会自动发生。行业调研的回归分析显示，AI 采用率每提高 25%，交付吞吐量下降 1.5%、交付稳定性下降 7.2%^[19]，与前述个体层面的实验结果方向一致。两项研究共同指向同一结论：AI 工具的价值取决于工程上下文与研发流程的配套程度，而不是工具本身的先进程度。这也解释了为什么本项目的定位是工程上下文供给层，而不是又一个编码助手。

8.8.5 缩短大型迁移与适配周期

迁移与适配是本项目效益中最直接可观察的一项。命题面向的鸿蒙化场景具有明确的工作量特征：公开披露的信息显示，微信为适配鸿蒙成立了约 800 人的团队，约占其开发人员的一半^[13]；《鸿蒙生态人才白皮书 2025》测算未来三至五年鸿蒙生态新增人才需求将超过 100 万人^[14]。与之相对，原生鸿蒙终端设备已超过 6,600 万台，注册开发者超过 1,100 万^[15]。迁移侧的工作量无法仅靠增加人力解决，工程化提效因此

从可选项变为必要项。

迁移的难点集中在依赖恢复而非语法转换。跨语言依赖无法由通用文本匹配或单一语言的语义分析完整恢复：专门的跨语言指针分析工作通过结构化的跨语言分析，对 JNI 方法调用与对象创建的解析召回率达到 97.7% 与 96.9%^[16]，这一水平需要专门的分析能力才能达到。RECAST 已接入明确 JNI 导出、N-API 注册链以及同模块 KMP 声明与实现的匹配规则，在 VS Code 工程中提取了 117,420 条导入、导出及工程引用关系；修改 N-API 注册信息后，增量分析能够更新相关调用关系，结果与相同源码的全量构建一致。对于缺少注册、模块不匹配或存在多个候选目标的情况，系统保留未解析或歧义状态，不生成不可靠的连接。

迁移效益的测算形式为：

$$\Delta V_4 = M \times L \times \tau \times v \quad (8.4)$$

其中 M 为年迁移模块或应用数， L 为单模块平均投入（人月）， τ 为周期压缩比例， v 为人月成本。 M 与 L 取自企业实际任务， τ 由迁移任务的试点对照标定。除直接的人力节省外，迁移周期缩短还带来业务上线提前的间接收益，该部分在试点中按业务口径单独评估，不计入本节的人力工时测算。

8.8.6 平台投入成本与净收益

前述效益均以平台自身的可运行和可维护为前提，因此需要同时给出投入侧数据。本节的投入数据全部来自真实工程实测，不依赖假设。

在 VS Code 真实源码上，系统完成 7,708 个文件、2,356,695 物理行的扫描解析、结构持久化与检索投影，三个阶段分别耗时 385.157 秒、868.734 秒和 784.169 秒，合计 2,038.06 秒，约 34.0 分钟。按此折算约为 865 秒每百万行、14.4 分钟每百万行；线性外推到千万行工程，全量建库约需 2.4 小时。建库是一次性投入，后续按变化影响范围增量更新，已完成的分析结果可以复用。

资源占用方面，主进程峰值常驻内存为 1.386 GiB，JavaScript 堆峰值为 987.3 MiB，解析子进程峰值常驻内存为 128.3 MiB。该结果说明百万行级工程的建模可以在单台通用开发机或普通服务器上完成，不需要专用集群。

模型调用方面，大模型只参与离线的模块职责分析。在 7,702 个文件的主工程上，模块建模、结果发布与检索投影合计耗时 24.304 秒，仅消耗 24 次模型请求，形成 66 个模块和 6 个子系统。在线检索不调用大模型，使用本地索引与 384 维语义编码模型完成。

由此形成与逐任务探索截然不同的成本结构：平台成本集中在建库阶段且可被全部后续任务分摊，在线查询的边际模型成本接近于零；而依赖智能体逐文件探索的方案，成本随任务数量线性增长，且无法在任务之间复用。

表 8.6: 平台投入实测结果

测量项目	实测结果
全量建库耗时	2,038.06 秒（约 34.0 分钟）
单位规模建库成本	约 14.4 分钟 / 百万行
千万行工程折算	约 2.4 小时（一次性）
主进程峰值内存	1.386 GiB
解析子进程峰值内存	128.3 MiB
离线模块分析模型调用	24 次请求 / 7,702 个文件
在线查询模型调用	0 次

在上述投入水平下，平台的经济性取决于效益项的量级。表 8.7 汇总本节各效益项的指标口径与数据状态。

表 8.7: 经济效益汇总与数据状态

效益项	计量指标	数据状态	结论口径
定位与理解工时	人日 / 年	参数有外部基准，压缩比例待试点标定	按人力薪酬口径取保守值
重复开发避免	人日 / 年	重叠比例在目标库实测，复用比例待标定	不采用公开生态重复率
Agent 上下文成本	元 / 年	由试点对照的 token 计量直接换算	无需估算
迁移周期	天 / 模块	任务量与投入取企业实际，压缩比待标定	单独评估，不与工时相加
平台投入	见上表，已实测	建库、内存、模型调用均为实测值	可直接给出

其中平台投入一侧已经可以用真实工程数据回答：百万行级工程的一次性建库成本约为 34 分钟、单机内存峰值 1.386 GiB，在线查询不产生大模型调用。这意味着无论效益系数最终标定为多少，平台的**固定投入都是可预测且在单机范围内可控的**，而收益随任务量与团队规模线性增长。因此本节的结论不是给出一个具体金额，而是说明收益的结构、量级与标定路径。

8.8.7 测算验证方案与不确定性说明

本节给出的效益模型包含若干只能由企业试点确定的系数。为避免把待标定参数当作已知结论，团队将其单独列出，并给出对应的标定实验设计。

标定采用同任务、同模型、同硬件条件下的 A/B 对照。对照组使用通用检索与逐文件探索方式，实验组接入 RECAST 的任务检索与上下文构建接口；两组使用相同的自然语言任务集、相同的目标工程快照和相同的代码生成模型，仅改变工程上下文的获取方式。每项任务记录代码定位与分析工时、检索命中情况、Agent 交互轮数、输入与输出 token 数、编译与行为测试结果，以及人工修复的代码行数与工时。

在此基础上按四组指标计算：效率指标包括定位工时与任务总周期；质量指标包括检索必需代码覆盖率、编译通过率和行为用例通过率；成本指标包括模型 token 消耗与折算金额；复用指标包括成功复用的模块数与避免的重复实现工作量。对照实验同时记录未完成任务及其原因，不将失败样本排除在统计之外。

待标定系数与验证方式对应如下：压缩比例 β 、可识别复用比例 γ 与残余适配比例 ρ 、周期压缩比 τ 由上述 A/B 对照直接测得；单任务模型成本 C_0 与 C_1 由 token 计量与模型定价直接换算。所有系数给出置信区间与样本量，而不是单一取值。

需要说明三项限制。第一，现有实测结果覆盖百万行级工程与函数级及类级任务，千万行规模与完整业务模块迁移仍在验证中，因此线性外推的结论应视为量级估计。第二，部分实测数据来自不同语料与不同配置轮次，跨语料的直接比较需在统一任务集上复核后才能作为结论引用。第三，效益模型中的外部基准值来自公开研究，其统计口径与国内大型工业软件企业未必完全一致，因此在正式测算中统一采用保守取值，并在结论中说明口径。

本节数据分为三类：外部权威基准、平台实测值与待企业标定值。外部基准用于刻画行业现状，平台实测值用于刻画系统当前能力，待标定值均已列出标定方法。团队在数据整理过程中剔除了若干流传广泛但无法溯源的说法，以避免以未经核实的数字支撑结论。所有平台实测数据均可在项目日志中复现。

表 8.8: 8.8 节数据来源

编 号	来源	在本节中的作用
[1]	Lientz, Swanson & Tompkins. Characteristics of Application Software Maintenance. <i>CACM</i> 21(6):466–471, 1978. DOI: 10.1145/359511.359522	维护工作构成（完善性 56% / 适应性 23% / 纠错性 21%）

表 8.8 8.8 节数据来源（续）

编号	来源	在本节中的作用
[2]	Sadowski, Stolee & Elbaum. How Developers Search for Code: A Case Study. <i>ESEC/FSE</i> 2015:191–201. DOI: 10.1145/2786805.2786855; Stolee et al. 10 Years Later: Revisiting How Developers Search for Code. <i>FSE</i> 2025. DOI: 10.1145/3715774	代码检索是高频活动，动因含寻找可复用实现
[3]	华为投资控股有限公司《2025 年年度报告》	研发费用 1,923 亿元、研发员工约 11.4 万人；价值换算口径
[4]	Lopes et al. DéjàVu: A Map of Code Duplicates on GitHub. <i>OOPSLA</i> 2017, PACMPL 1(OOPSLA):84. DOI: 10.1145/3133908	GitHub 上约 70% 代码存在重复
[5]	Juergens et al. <i>ICSE</i> 2009. DOI: 10.1109/ICSE.2009.5070547; Rahman et al. <i>EMSE</i> 17(4–5). DOI: 10.1007/s10664-011-9195-3; Wagner et al. <i>SANER</i> 2016. DOI: 10.1109/SANER.2016.94	克隆的风险集中在”应同步却已分叉”
[6]	美国国防部 DTIC 公开文件 ADA431122; Poulin, Caruso & Hancock. <i>IBM Systems Journal</i> 32(4):567–594, 1993. DOI: 10.1147/sj.324.0567	NASA SEL 研发成本下降 55%; Boeing STS 返工下降 31%、ROI 7.75:1
[7]	Liu et al. Lost in the Middle: How Language Models Use Long Contexts. <i>TACL</i> 12:157–173, 2024. DOI: 10.1162/tacl_a_00638	中部信息准确率下降约 20 个百分点；扩展上下文版本并不更优
[8]	Hsieh et al. RULER: What’s the Real Context Size of Your Long-Context Language Models? <i>COLM</i> 2024. arXiv:2404.06654	有效上下文长度普遍远低于宣称长度

表 8.8 8.8 节数据来源（续）

编号	来源	在本节中的作用
[9]	Modarressi et al. NoLiMa: Long-Context Evaluation Beyond Literal Matching. <i>ICML</i> 2025, PMLR v267. arXiv:2502.05167	移除字面匹配后 32K 时跌至基线 50% 以下
[10]	METR. Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. 2025-07-10. arXiv:2507.09089	16 名资深开发者、246 个真实任务，耗时长 19%，主观认为快 20%
[11]	Xia et al. Agentless: Demystifying LLM-based Software Engineering Agents. arXiv:2407.01489	仓库级修复任务每实例约 0.34 美元的成本参照
[12]	Chen et al. LocAgent: Graph-Guided LLM Agents for Code Localization. <i>ACL</i> 2025. arXiv:2503.09089	图引导定位方案成本降低约 86%
[13]	腾讯高级副总裁蔡光忠公开披露，经 IT 之家、凤凰科技等报道	微信约 800 人团队适配鸿蒙，约占开发人员一半
[14]	《鸿蒙生态人才白皮书 2025》	未来 3–5 年鸿蒙生态新增人才需求超 100 万人
[15]	华为开发者大会 2026 官方发布	原生鸿蒙终端超 6,600 万台；注册开发者超 1,100 万
[16]	Interactive Cross-Language Pointer Analysis for Resolving Native Code in Java Programs. <i>ICSE</i> 2025. DOI: 10.1109/ICSE55347.2025.00075	JNI 方法调用/对象创建解析召回率 97.7% / 96.9%
[17]	Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. <i>NeurIPS</i> 2024. arXiv:2405.15793	开放式智能体方案的单实例成本对照

表 8.8 8.8 节数据来源（续）

编号	来源	在本节中的作用
[18]	SHERLOC: Structured Diagnostic Localization for Code Repair Agents. arXiv:2606.24820	定位类智能体轨迹的 token 消耗中位数 2.86 万
[19]	Google Cloud DORA. 2024 State of DevOps Report	AI 采用率每提高 25% 对应的吞吐量与稳定性变化

8.9 下一阶段开发与验证计划

下一阶段将围绕大仓建模、多语言依赖、任务检索和企业应用继续开发，并将功能完善与对应验证同步推进。

8.9.1 大仓建模与增量维护

面向更大规模、多仓库工程，继续完善结构记录与检索索引分片，减少单次加载和更新的数据量。增量维护将区分实现正文、公开接口和构建配置的变化，缩小模块重算范围，并扩展多个执行节点之间的任务调度与故障恢复。验证时采用不同规模的真实工程，记录建库时间、内存开销和局部更新时间。通过中断恢复与全量构建对照，检查更新后的模块、符号和依赖是否一致。

8.9.2 跨语言依赖分析

继续补充跨语言注册模式与构建配置解析，结合完整 SDK、编译参数和平台工程完善符号绑定。重点覆盖复杂 JNI 注册、动态注册、平台源集及跨构建目标调用，并扩展 ArkUI 专用语法的解析支持。验证样例将覆盖重载、宏和不同平台实现等情况，检查接口声明、注册入口与目标实现能否正确连接。对于无法确定的关系，保留未解析或歧义状态；注册与配置变化后，检查旧连接是否清除、新关系是否正确建立。

8.9.3 检索与上下文优化

检索任务将从函数定位扩展到模块修改与跨文件任务，继续优化模块代表源码选择、候选排序和依赖读取。通过有标注的任务集，检查模块路由、候选重排与上下文选择能否找到任务所需实现，并减少无关读取和重复内容。上下文验证将关注完整函

数、关键依赖及后续补查之间的衔接，使返回结果能够支持连续分析。在固定硬件、源码版本和缓存条件下测量平均响应时间，结合实际返回的源码与依赖检查任务结果。

8.9.4 产品流程与企业验证

完善迁移审阅页面与任务执行服务的联调，使用户能够连续查看参考代码、修改计划、文件差异和验证结果。围绕这一流程补充回归用例，检查工程范围、任务状态和验证结果是否保持一致。随后结合企业真实工程开展代码复用、修改与迁移试用，通过编译和行为测试检查适配结果，并记录失败任务及人工修复情况。将试用中发现的问题转化为回归用例，持续完善依赖分析、上下文交付和产品交互。

第九章 项目团队与组织分工

9.1 团队总体情况

ReCAST 项目团队由计算机科学与技术、大数据与数据科学、人工智能等多学科背景成员组成，形成了覆盖软件工程、人工智能、数据分析、系统开发和产品实现的复合型研发团队。能够从技术研发、数据处理、产品设计和项目组织多个维度推动项目实施。

项目团队采用“教师指导、学生主体研发”的组织模式。指导教师负责技术路线把关、需求分析和项目方向指导；学生团队承担核心技术研发、系统实现、实验验证和产品展示等具体工作，确保项目能够从技术方案设计推进到实际系统落地。

9.2 团队组织与任务分工

项目团队按照系统研发流程进行组织，围绕“代码理解与索引—依赖分析—智能检索—系统实现—测试验证”的技术链路开展协同研发。

9.2.1 指导教师

项目指导教师具有软件工程、工业智能体及智能系统方向的科研积累，在相关领域开展长期研究工作，为项目技术路线设计、需求分析和产品落地提供指导。

表 9.1: 项目指导教师信息

免冠照	姓名	专业方向	简介
照片待补充	杨振 助理教授	软件工程、程序分析与智能化软件开发	杨振，山东大学计算机科学与技术学院助理教授，长期从事软件工程、程序分析与智能化软件开发相关研究。 近 5 年来，在 FSE、ASE、ISSTA、TSE、TOSEM 等软件工程领域国际顶级期刊和会议发表论文 30 余篇，其中一作/通讯发表 CCF-A 类论文 8 篇，CCF-B 类论文 12 篇。入选 2025 年度山东省青年科技人才托举工程、山东省引进海外科技人才。 在本项目中，负责核心技术路线设计与技术方案指导，围绕超大规模代码理解、代码索引构建、跨语言依赖分析以及智能检索等关键技术开展指导。
照片待补充	孙景昊 教授	工业智能体、自动驾驶运行时软件技术、实时调度与优化理论	孙景昊，山东大学计算机学院教授，主要研究方向包括工业智能体、自动驾驶运行时软件技术以及实时调度与优化理论。 主持国家自然科学基金重大研究计划培育项目、面上项目、青年项目、省自然科学基金项目以及华为胡杨林基金等多项科研项目。 近 5 年来，以第一作者/通信作者身份在 IEEE Transactions on Computers、IEEE Transactions on CAD、ACM Transactions on Embedded Computing Systems、RTSS、RTAS、EMSOFT、DAC 等国际期刊和会议发表论文 18 篇，其中 TOP 类期刊论文 6 篇，CCF-A/B 类论文多篇。 在本项目中，负责结合企业命题开展需求解析，指导项目产品定位、应用场景设计以及整体落地方向规划。
照片待补充	李腾龙 辅导员	学生团队建设与项目过程指导	负责学生团队建设与项目过程指导，关注团队协作、任务推进和成果凝练，协助学生提升项目组织能力和工程实践能力，为项目持续推进提供支持。

9.2.2 学生团队

项目学生团队由计算机科学、数据科学、智能科学与技术等专业成员组成，围绕代码理解、智能检索、系统开发和实验验证等方向开展协同研发。

表 9.2: 学生团队成员信息

免冠照	姓名	专业方向	简介
照片待补充	吕曦冉	大数据与数据科学	项目负责人，负责项目整体推进、技术方案协调和系统架构设计。参与需求分析、技术路线规划以及项目成果组织工作，负责推动代码理解、智能检索和系统实现多个模块协同开发。曾参与软件工程领域相关科研工作，发表多篇 CCF-A 类论文；获院级优秀学生称号，并获得 2025 年潍柴动力奖学金。
照片待补充	唐程	计算机科学	负责超大规模代码仓索引构建与工程数据处理相关研发。参与代码解析流程设计、代码结构表示以及大规模工程数据组织，为系统代码理解能力提供基础支撑。 具备人工智能、机器学习与医学信息处理研究经历，参与 AI4Health 科研项目，开展医学数据分析、机器学习建模及智能算法研究。曾获全国大学生数学建模竞赛山东赛区三等奖、山东大学优秀学生二等奖。
照片待补充	赵旭东	计算机科学与技术	负责软件工程分析与代码依赖建模相关工作。参与程序结构分析、模块关系恢复以及跨模块依赖信息构建。 具备计算机系统方向实践经验，曾参加全国大学生计算机系统能力大赛操作系统设计赛 OS 功能挑战赛道并获得三等奖。
照片待补充	张汇智	人工智能	负责智能检索算法与语义匹配相关研发。参与自然语言需求理解、向量检索以及候选代码排序优化。 具备人工智能技术应用实践经验，目前正在华为开展 AI 技术应用相关实习，曾获昇腾算子高级赛道二等奖。
照片待补充	赵瀚兴	计算机科学与技术	负责系统后端开发和服务接口设计。参与核心模块集成、系统服务部署以及平台运行环境搭建。 具有大语言模型与软件工程交叉实践经验，参与基于大语言模型的跨编程语言代码自动生成项目，结合自动化测试与迭代修复提升代码生成可靠性。

续下页

表 9.2 学生团队成员信息（续）

免冠照	姓名	专业方向	简介
照片待补充	王润鲁	计算机科学与技术	负责系统功能实现与模块整合工作。参与平台工程开发、功能联调以及整体系统稳定性优化。具备数据库系统与工程开发实践经验，曾获全国大学生计算机系统能力大赛数据库管理设计赛全国一等奖，并获得 OceanBase 数据库创新设计赛三等奖。
照片待补充	张博文	计算机科学与技术	负责检索流程优化与实验评估工作。参与候选召回、排序策略设计以及检索效果分析。具备计算机系统方向竞赛与工程实践经验，曾获 2026 年全国大学生计算机系统能力大赛操作系统设计赛全国总决赛三等奖，并获得 2025 年国家奖学金。
照片待补充	彭煜程	大数据与数据科学	负责实验数据处理、测试方案设计以及实验结果分析。参与系统性能评估和验证流程建设。计算机菁英班成员，具备数据分析与工程实践能力，获院级优秀学生称号。
照片待补充	马丽	公信	负责项目需求整理、文档组织和产品展示支持。参与项目材料整理、应用场景分析以及团队协作工作。具备产业实践与产品支持经验，曾在山东产业技术研究院实习，获得机器视觉工程师证书；曾获大学生数学建模山东省三等奖、新生杯山东大学校级铜奖，并获山东大学社会实践优秀团队及优秀报告等荣誉。
照片待补充	吴东贤	公信	负责项目资料整理、测试支持和应用场景调研工作，协助完成项目展示与成果总结。具备产品设计与应用研究实践经验，曾独立完成英语拼写练习平台从 0 到 1 的产品落地，并参与网络舆论极化演化规律与干预策略研究；获得 2025 年山东大学港澳及华侨奖学金。

续下页

表 9.2 学生团队成员信息（续）

免冠照	姓名	专业方向	简介
照片待补充	耿子凡	财政学	负责财税政策评估与金融量化分析相关研究工作。参与政策效应量化测算、场景数据建模以及评价指标体系构建，为项目的政策适配分析与业务落地提供决策支撑。 具备财税政策研究、财务审计实践经验，参与中国政府预算改革发展年度报告等课题项目；参与全国人大常委会预算工委税负结构调研，开展宏观税负分析，相关研究建议被官方采纳并获感谢信回复。曾获第 16 届“工行杯”全国大学生金融科技创新大赛山东省赛一等奖、山东大学优秀毕业生等荣誉。

9.3 师生协同与项目组织

项目采用“需求驱动、技术牵引、学生主体”的协同研发机制。

在项目推进过程中，指导教师负责从产业需求和技术发展角度进行方向把控，参与关键技术方案讨论、阶段成果评审以及产品定位优化；学生团队负责具体研发实施，包括需求调研、算法设计、系统开发、实验测试和 Demo 验证。

团队内部按照技术模块建立协作机制。针对超大规模代码理解、跨语言依赖分析、智能检索与上下文构建等核心问题，各研发小组定期同步开发进展，通过代码评审、实验分析和方案讨论不断优化系统设计。

同时，项目以企业命题需求为牵引，通过需求分析、方案设计、系统开发和效果验证形成闭环迭代流程。企业场景反馈用于指导功能完善和技术优化，确保项目研发方向与实际产业需求保持一致。

通过上述组织模式，团队实现了科研能力、工程实践和产业需求的有效结合，为项目持续研发和后续落地提供组织保障。